International Cybersecurity and Digital Forensics Academy (ICDFA)

Lab 1B: PC System Fundamentals, Disk Sectors and Partition Analysis

Lab Report

Student name	Sarah Andrew Il
Registration number	C11/26/PCDFIR/17032
Course code	CIP-B101
Lab title	Lab 1B: PC System Fundamentals, Disk Sectors and Partition Analysis
Date	6th November 2026
Instructor Name	Abdullahi Umar Abbas

1. Introduction

The purpose of this lab is to build a basic system profile of a computer in the way a junior digital forensic trainee would before an investigation. I identified the hardware and operating system, looked at running processes, and mapped where digital evidence can be found in each system layer.

I also inspected the disk and its partition table using read-only commands, calculated partition sizes from sector counts, and checked the boot mode and file systems. The aim was to observe, record, calculate and verify, without changing anything on the machine.

Only read-only commands were used (cat, lscpu, free, lsblk, blkid, df, ps, fdisk -l, parted -l and xxd). No partition was created, deleted, resized, formatted or written to.

2. Tools Used

Item	Details
Environment	Kali GNU/Linux Rolling 2026.1 running as a virtual machine in Oracle VirtualBox (Linux/Kali VM option of the lab)
System commands	cat /etc/os-release, lscpu, free -h
Storage / partition commands	lsblk, lsblk -o, lsblk -f, sudo fdisk -l, sudo parted -l, sudo blkid, df -Th
Process command	ps -eo pid,comm,%cpu,%mem --sort=%cpu | head -n 15
Hex view of MBR	sudo xxd -l 512 /dev/sda (reads the first 512 bytes only)
Boot mode check	if [-d /sys/firmware/efi]; then echo "UEFI"; else echo "BIOS/Legacy"; fi
Calculator	Python 3 (python3 -c) and manual formulas
Screenshots	Taken inside the VM terminal

The Windows tools in the manual (msinfo32, systeminfo, wmic, Disk Management, DiskPart) were replaced by their Linux equivalents because the practical was done on a Kali Linux VM. Instead of WinHex (step D3), I used the Linux tool xxd to view sector 0 in hex.

3. System Specification Evidence (Part A)

A1. Basic system specification

I used cat /etc/os-release for the operating system, lscpu for the processor, free -h for memory, and a short test of /sys/firmware/efi for the boot mode (shown in Part E).

```
┌──(sarah㉿kali)-[~]
└─$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2026.1"
VERSION="2026.1"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"
```

Figure 1: Operating system information (cat /etc/os-release)

```
ANSI_COLOR="1;31"

┌──(sarah㉿kali)-[~]
└─$ lscpu
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             39 bits physical, 48 bits virtual
  Byte Order:                Little Endian
CPU(s):                      1
  On-line CPU(s) list:       0
Vendor ID:                   GenuineIntel
  Model name:                Intel(R) Core(TM) i5-7200U CPU @ 2.50GHz
    CPU family:              6
    Model:                   142
    Thread(s) per core:      1
    Core(s) per socket:      1
    Socket(s):               1
    Stepping:                9
    BogoMIPS:                5423.99
```

Figure 2: Processor information (lscpu)

```
┌──(sarah㉿kali)-[~]
└─$ free -h
               total        used        free      shared  buff/cache   available
Mem:           1.9Gi       865Mi       152Mi        19Mi       1.1Gi       1.1Gi
Swap:          1.6Gi          0B       1.6Gi
```

Figure 3: Memory information (free -h)

Item	Observed value
OS name	Kali GNU/Linux Rolling
Version	2026.1 (kali-rolling, based on Debian)
System type / architecture	x86_64, 64-bit (CPU supports 32-bit and 64-bit modes)
Processor	Intel(R) Core(TM) i5-7200U @ 2.50 GHz (1 CPU, 1 core, 1 thread given to the VM)
Installed memory (RAM)	1.9 GiB total, 865 MiB used, 1.1 GiB available
Swap space	1.6 GiB
Virtualization	Full virtualization (hypervisor shown as KVM by lscpu) inside VirtualBox
BIOS mode	BIOS/Legacy (no /sys/firmware/efi folder found)
Secure Boot state	Not applicable. Secure Boot is a UEFI feature and this system boots in BIOS/Legacy mode.

Important observation: because this is a virtual machine, the hardware is virtual. In a real investigation, I would also need to record the host machine and the hypervisor, because evidence can exist in both places.

A2. Storage information

The lsblk -o command shows the disk model, size and file systems. The sudo fdisk -l command shows the sector details of the disk (Figure 4 also shows the boot mode test and plain lsblk output at the top).

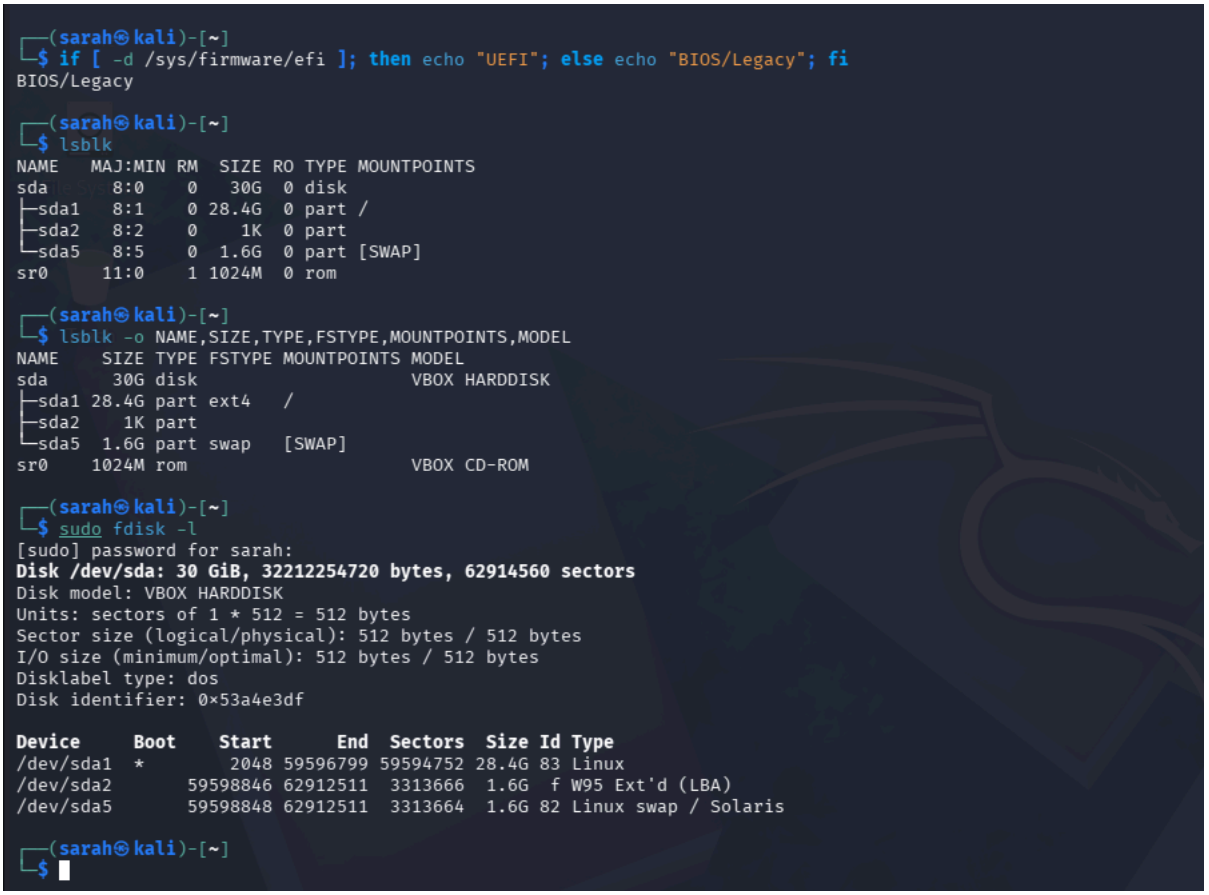


Figure 4: Storage information: BIOS test, lsblk, lsblk -o and sudo fdisk -l

Device	Size	Type	File system	Mount point / note
sda	30 GiB	disk	-	VBOX HARDDISK (virtual disk)
sda1	28.4 GiB	partition (primary)	ext4	Mounted on / (root), boot flag set
sda2	1K shown by lsblk	partition (extended)	none	Container for logical partitions
sda5	1.6 GiB	partition (logical)	swap	[SWAP]
sr0	1024 MiB	rom	-	VBOX CD-ROM

A3. Evidence sources mapped to system layers

User activities = Applications + Operating System + Disk/Memory. Every action a user takes passes through several layers, and each layer can keep its own record.

Layer / Source	Example evidence	Where it may be found	Why it matters
User layer	Opened files, searched terms, typed usernames	Recent files, browser history, shell history (~/.bash_history)	Links a person's action to system activity
Application layer	Browser cache, Office document metadata, chat logs	Application folders and databases (e.g. ~/.mozilla for Firefox)	Shows which software created or used the evidence
Operating system layer	Logs, installed programs, user accounts, login times	Linux /var/log, /etc/passwd; Windows event logs and registry	Shows system events and account activity
Disk / file system layer	Deleted files, file fragments, partitions, timestamps	File system structures and unallocated space	Allows file recovery and timeline building
Memory / process layer	Running malware, PIDs, network connections, passwords in RAM	RAM capture, process list (ps, /proc)	Shows what the system was doing live; lost when powered off
Firmware / hardware layer	Boot order, boot mode, device serial numbers	BIOS/UEFI settings, partition table, MBR	Explains how the system started and what devices were attached

Why evidence may exist in more than one layer: One user action usually leaves traces in many places. For example, if someone opens a file from a USB drive, the application may save it in a recent-files list, the operating system may log that the drive was plugged in, the disk may keep a timestamp for the file, and the memory may still hold the file's content. If a suspect deletes evidence from one layer, it may still be found in another. Matching traces across layers also makes the evidence more reliable in court.

4. Process Observation (Part B)

B1. Running processes

I used the Linux ps command (the equivalent of tasklist or Task Manager) to list running processes with their PID, name, CPU use and memory use.

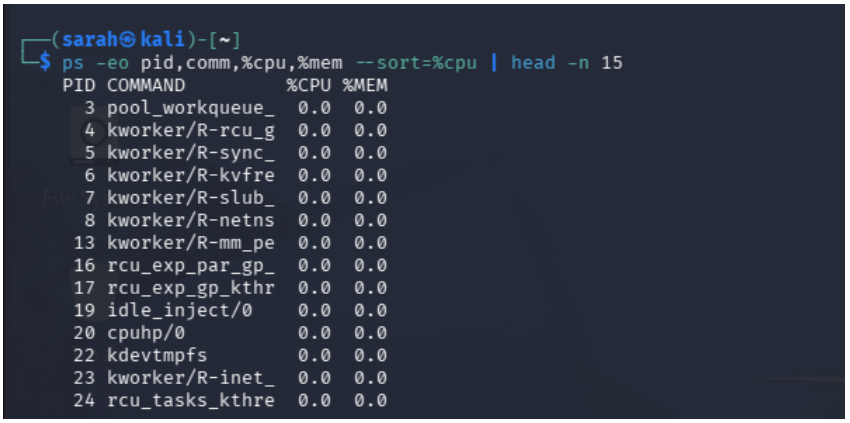


Figure 5: Running processes with PIDs (ps -eo pid,comm,%cpu,%mem --sort=%cpu | head -n 15)

Process name	PID	Type	Likely purpose
pool_workqueue_ (name cut off in the output)	3	Kernel thread	Part of the Linux kernel's work queue system, which runs small background jobs for the kernel
cpuhp/0	20	Kernel thread	Handles CPU hotplug, meaning the kernel adding or removing a CPU core. The 0 is the CPU number
kdevtmpfs	22	Kernel thread	Manages the /dev folder, where device files (such as disks) are created

All of the processes shown are kernel threads that were idle at that moment, so they show 0.0% CPU and memory use. They are shown by name only because ps -eo comm prints the short command name.

B2. Program, process and thread

- **Program:** a file stored on the disk that contains instructions, for example /usr/bin/firefox. It does nothing while it is only stored.
- **Process:** a program that is running. The operating system gives it a PID (process ID), its own memory space and resources. One program can run as many processes.
- **Thread:** a smaller unit of work inside a process. Threads in the same process share the same memory, and each thread can run a different task at the same time. My lscpu output shows 1 thread per core, which means the CPU of this VM can run one thread at a time.

Why process information is useful in forensics: During malware analysis or live forensic triage, the list of running processes shows what is active right now. An investigator can look for unknown or oddly named processes, processes using a lot of CPU, or processes that started from strange locations. The PID helps link a process to its network connections, open

files and parent process. This information exists only in memory, so it disappears when the computer is shut down. This is why it is collected early during live response.

5. Disk Sector and Size Calculations (Part C)

Key formula: Disk or partition size = number of sectors x bytes per sector. To convert bytes to GB, divide by 1024 x 1024 x 1024.

Note on units: dividing by 1024 three times gives gibibytes (GiB). The manual calls this GB, so I use the same convention, and Linux tools also print it as 'G'. Disk makers and the parted tool use 1000 instead of 1024, which gives slightly bigger numbers. I explain this in Part G.

C1. Slide calculation

Given: sectors per track = 400, heads = 12, cylinders = 17000, bytes per sector = 512.

Item	Formula / working	Answer
Total sectors	400 x 12 x 17,000	81,600,000 sectors
Total bytes	81,600,000 x 512	41,779,200,000 bytes
Kilobytes	41,779,200,000 / 1024	40,800,000 KB
Megabytes	40,800,000 / 1024	39,843.75 MB
Gigabytes	39,843.75 / 1024	38.91 GB

I also checked my Python calculator with a simple test: 1,048,576 sectors x 512 / 1024^3 = 0.5 GB (that is 512 MB).



*Figure 6: Python used as a calculator (1048576*512/1024**3 = 0.5)*

C2. My own disk and partitions

The values come from the partition table shown by sudo fdisk -l (Figure 4). The sector size is 512 bytes (logical and physical). The disk has 62,914,560 sectors in total.

Partition	Start sector	End sector	Sector count	Sector size	File system / type
/dev/sda1	2048	59,596,799	59,594,752	512 bytes	ext4 (primary, boot)
/dev/sda2	59,598,846	62,912,511	3,313,666	512 bytes	Extended (container)
/dev/sda5	59,598,848	62,912,511	3,313,664	512 bytes	Linux swap (logical)

Sector count check (end - start + 1):

- sda1: 59,596,799 - 2,048 + 1 = 59,594,752 (matches fdisk)
- sda2: 62,912,511 - 59,598,846 + 1 = 3,313,666 (matches fdisk)
- sda5: 62,912,511 - 59,598,848 + 1 = 3,313,664 (matches fdisk)

Size calculations (step by step):

Partition 1 (sda1)

- Bytes = 59,594,752 x 512 = 30,512,513,024 bytes
- In GB = 30,512,513,024 / 1024 / 1024 / 1024 = 28.42 GB

Partition 2 (sda2, extended)

- Bytes = 3,313,666 x 512 = 1,696,596,992 bytes
- In GB = 1,696,596,992 / 1024 / 1024 / 1024 = 1.58 GB (1,618 MB)

Logical partition inside partition 2 (sda5, swap)

- Bytes = 3,313,664 x 512 = 1,696,595,968 bytes
- In GB = 1,696,595,968 / 1024 / 1024 / 1024 = 1.58 GB

Whole-disk check

- 62,914,560 x 512 = 32,212,254,720 bytes = 30.00 GB. This matches the '30 GiB, 32212254720 bytes' line printed by fdisk.
- Layout: 2,048 sectors before sda1 (1 MiB gap) + 59,594,752 (sda1) + 2,046 unused sectors + 3,313,666 (sda2) + 2,048 unused sectors at the end = 62,914,560. All sectors are accounted for.

6. Safe Partition Table Inspection (Part D)

I used the Linux/Kali method (D2) and a hex view of sector 0 (D3). Only read-only commands were run. The lsblk, lsblk -o and sudo fdisk -l outputs are in Figure 4. The sudo parted -l output is below.

```
┌──(sarah㉿kali)-[~]
└─$ sudo parted -l
Model: ATA VBOX HARDDISK (scsi)
Disk /dev/sda: 32.2GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number  Start   End     Size    Type      File system     Flags
 1      1049kB  30.5GB  30.5GB  primary   ext4            boot
 2      30.5GB  32.2GB  1697MB  extended                  lba
 5      30.5GB  32.2GB  1697MB  logical   linux-swap(v1)  swap
```

Figure 7: Partition table listing (sudo parted -l)

Findings from the partition table:

- Disk /dev/sda is a VBOX HARDDISK of about 30 GiB (32.2 GB). Sector size is 512 bytes logical and physical.

- **Partition table type:** fdisk shows 'Disklabel type: dos' and parted shows 'Partition Table: msdos'. This means the disk uses MBR, not GPT. Together with the BIOS/Legacy boot result, this is what I expected.
- Three entries are listed: sda1 (primary, ext4, boot flag), sda2 (extended) and sda5 (logical, swap). sda5 sits inside sda2, which is why sda2 and sda5 have almost the same size.
- The disk identifier 0x53a4e3df matches the PARTUUID prefix (53a4e3df-01, 53a4e3df-05) shown by blkid in Part F.

D3. Hex view of the MBR (sector 0)

WinHex is a Windows tool, so I used the Linux command sudo xxd -l 512 /dev/sda instead. It only reads the first 512 bytes of the disk (sector 0) and shows them in hex, so nothing is changed.



Figure 8: First 512 bytes of /dev/sda (sudo xxd -l 512 /dev/sda)

Part of the MBR	Offset	Bytes seen	Meaning
Boot code	0x000 - 0x1B7	eb 63 90 ... plus readable text 'GRUB', 'Geom',	Boot loader code. The text shows GRUB is installed in the MBR

Part of the MBR	Offset	Bytes seen	Meaning
		'Hard Disk', 'Read', 'Error'	
Disk signature	0x1B8	df e3 a4 53	Stored in reverse order (little-endian), this is 0x53a4e3df, the same as the disk identifier in fdisk
Partition entry 1	0x1BE	80 ... 83 ... 00 08 00 00 | 00 58 8d 03	Boot flag 80 (active), type 83 (Linux). Start = 0x800 = 2,048. Size = 0x038D5800 = 59,594,752 sectors
Partition entry 2	0x1CE	00 ... 0f ... fe 67 8d 03 | 02 90 32 00	Not active, type 0f (extended LBA). Start = 0x038D67FE = 59,598,846. Size = 0x00329002 = 3,313,666 sectors
Partition entries 3 and 4	0x1DE, 0x1EE	all 00	Not used
Boot signature	0x1FE	55 aa	Marks a valid MBR

These numbers match the fdisk output exactly (start 2048 / 59,594,752 sectors for sda1 and start 59,598,846 / 3,313,666 sectors for sda2). This confirms from the raw disk bytes that the disk uses MBR with only two entries in the main table. The logical partition sda5 is not listed here because logical partitions are described inside the extended partition, not in the MBR.

7. Boot Process, MBR and GPT (Part E)

E1. How a PC boots

1. **Power-on:** the power supply starts and the CPU begins running firmware code.
2. **POST (Power-On Self-Test):** the firmware checks that the CPU, memory and basic hardware work.
3. **BIOS/UEFI firmware and CMOS settings:** the firmware reads its saved settings (kept in CMOS/NVRAM) such as the boot order, and chooses the boot device.
4. **Boot records:** on a BIOS/MBR disk the firmware reads the MBR in sector 0. On a UEFI/GPT disk it reads the GPT and finds the EFI System Partition boot entry.
5. **Boot loader / boot manager:** a small program (for example GRUB on Kali, or Windows Boot Manager) starts, may show an operating system menu, and loads the kernel.
6. **Kernel loading:** the kernel is loaded into memory, starts the hardware drivers and mounts the root file system (on this system /dev/sda1).

7. **Operating system control:** the kernel starts the first process and system services. Logs and user activity begin to build up from here.

Stage	What happens	Forensic relevance
Power-on and POST	Hardware is started and checked	Hardware faults or tampering can affect access to evidence
BIOS/UEFI	Settings and boot order are applied	Boot mode explains partition style and boot artifacts
MBR/GPT or EFI boot	Bootable partition or EFI entry is found	Boot records show boot configuration
Boot loader	OS is selected and kernel is loaded	Can reveal multi-boot or hidden operating systems
Kernel and OS	System fully starts	Logs and user activity begin

Boot mode of my system: The test for /sys/firmware/efi returned BIOS/Legacy. The folder only exists when a system booted in UEFI mode, so my system uses BIOS/Legacy boot. This agrees with the MBR (dos) partition table found in Part D.



Figure 9: Boot mode evidence: system reports BIOS/Legacy

E2. MBR compared with GPT

Feature	MBR (Master Boot Record)	GPT (GUID Partition Table)
Age and design	Older system from the 1980s, used with BIOS	Modern system, normally used with UEFI
Size limit	About 2 TB per disk; at most 4 primary partitions (or 3 primary plus an extended partition with logical partitions)	Very large disks (slide notes about 9.4 ZB); up to 128 partitions in most systems
Where partition info is kept	In sector 0 together with the boot code	In a primary GPT header and partition entries at the start, and a backup copy at the end of the disk
Forensic issue	All key information is in one sector, so damage there can stop the partitions from being read	The backup copy helps recovery and checking. A protective MBR in sector 0 may also be present
Seen on my system	Yes (dos / msdos, with a primary, extended and logical partition)	No

8. File System Identification (Part F)

I used lsblk -f, blkid and df -Th to identify the file systems.

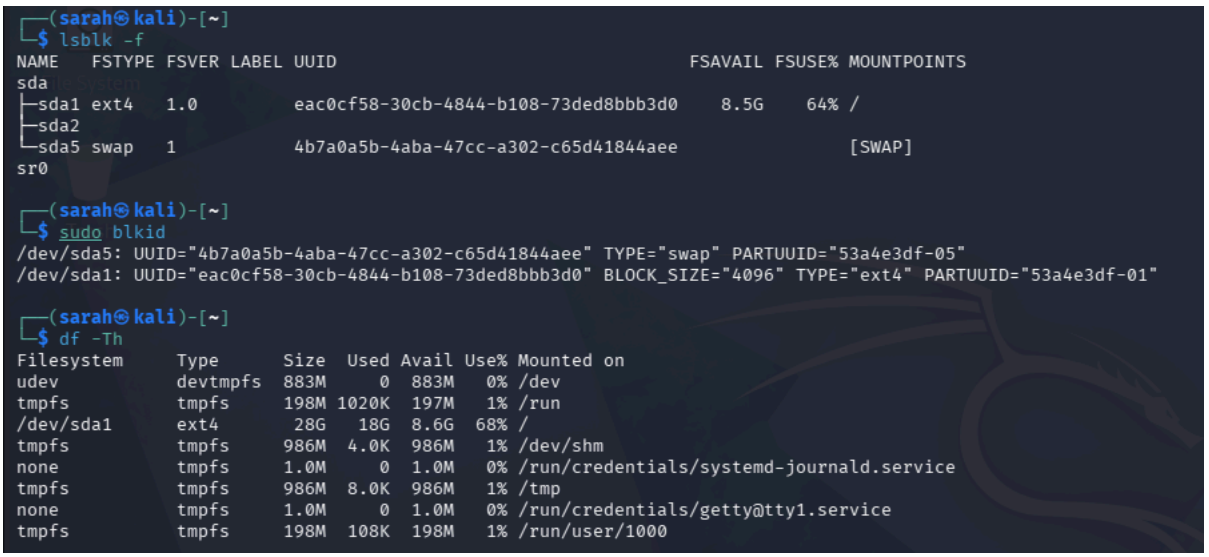


Figure 10: File system evidence (lsblk -f, sudo blkid, df -Th)

Partition	File system	Mount	Details from the output
/dev/sda1	ext4	/ (root)	Version 1.0, block size 4096, UUID eac0cf58-30cb-4844-b108-73ded8bbb3d0, 8.5G available (lsblk -f shows 64% used, df shows 68%)
/dev/sda5	swap (v1)	[SWAP]	UUID 4b7a0a5b-4aba-47cc-a302-c65d41844aee. Used by Linux as extra virtual memory
/dev/sda2	none	-	Extended partition container, no file system of its own

Why the file system matters in digital forensics: The file system decides how files are named, stored, found and deleted. It also decides what extra information is kept, such as timestamps (created, modified, accessed), owners and permissions. Knowing the file system tells the investigator which tools to use and where to look for deleted files and hidden data. For example, ext4 stores file details in inodes and keeps a journal that can help with timeline analysis, while NTFS uses the Master File Table (MFT). The swap partition has no normal file system, but it can hold pieces of memory (for example passwords or documents) that were moved out of RAM, so it is also a source of evidence.

File system	Common OS / usage	Forensic importance
FAT/FAT32	Removable media and older systems	Simple structure; often used on USB drives
NTFS	Modern Windows	Has MFT entries, timestamps, permissions and a journal
ext family (ext4)	Linux (this system)	Inode-based metadata; supports timeline analysis

File system	Common OS / usage	Forensic importance
XFS / JFS / btrfs	Linux/Unix-like systems	Advanced features can affect recovery and metadata reading
HFS+	Older macOS	Important in Mac investigations; support has date limits

9. Final Practical Assignment (Part G)

Task: Verify the size of partitions 1 and 2 using the partition table, record the steps, and calculate each size in GB.

Verification steps

1. Ran sudo fdisk -l (read-only) to display the partition table of /dev/sda. Sector size = 512 bytes (Figure 4).
2. Recorded the start, end and sector count of each partition from the Start, End and Sectors columns.
3. Checked each sector count with the formula end - start + 1 (Part C2).
4. Calculated size in bytes = sectors x 512, then converted to GB by dividing by 1024 three times.
5. Compared the results with the sizes reported by the operating system (lsblk, df -h) and by parted (Figures 4, 7 and 10).

Task	Student notes / answer
Identify partition 1	/dev/sda1: primary partition, boot flag set, type 83 Linux, ext4 file system, mounted on / (Figures 4 and 7)
Partition 1 sector count	Start = 2,048; End = 59,596,799; Sectors = 59,594,752
Calculate partition 1 size	59,594,752 x 512 = 30,512,513,024 bytes. 30,512,513,024 / 1024 / 1024 / 1024 = 28.42 GB
Identify partition 2	/dev/sda2: extended partition (type f, W95 Ext'd LBA). It holds the logical partition /dev/sda5 (Linux swap) (Figures 4 and 7)
Partition 2 sector count	Start = 59,598,846; End = 62,912,511; Sectors = 3,313,666 (logical partition sda5 inside it: start 59,598,848, sectors 3,313,664)
Calculate partition 2 size	3,313,666 x 512 = 1,696,596,992 bytes. 1,696,596,992 / 1024 / 1024 / 1024 = 1.58 GB (sda5: 3,313,664 x 512 = 1,696,595,968 bytes = 1.58 GB)
Compare with OS-reported size	Partition 1: fdisk and lsblk show 28.4G, df -h shows 28G (used 18G, available 8.6G), parted shows 30.5GB. Partition 2: fdisk shows 1.6G, lsblk shows 1.6G for sda5 but only 1K for sda2, parted shows 1697MB

Task	Student notes / answer
Explain difference	See below

Explanation of the differences

- **GB vs GiB:** My calculation divides by 1024 three times, which gives 28.42 (GiB). The tool lsblk and fdisk round it to 28.4G. The parted tool divides by 1000 instead, so the same 30,512,513,024 bytes appears as 30.5GB. Likewise 1,696,596,992 bytes is 1.58 GiB but 1.70 GB (1697MB) in parted. The size is the same, only the unit is different.
- **df shows a smaller size (28G):** df reports the file system, not the partition. The ext4 file system uses part of the partition for its own structures (inode tables, journal and other metadata), so the usable size is a little smaller. df also rounds the value, and by default ext4 keeps some blocks reserved for the system, which is why Used (18G) plus Avail (8.6G) is less than the total.
- **sda2 shows 1K in lsblk:** sda2 is an extended partition, which is only a container. The Linux kernel reports it as 2 sectors (1 KiB), while fdisk shows the full range of sectors it covers (3,313,666). The real space is used by the logical partition sda5 inside it. The 2-sector difference between sda2 (3,313,666) and sda5 (3,313,664) is at the start of the extended partition, where the extended boot record and alignment gap normally sit.
- **Rounding:** Sizes shown by the tools are rounded to one decimal place, so small differences (28.42 vs 28.4) are normal.

Conclusion of the verification: my manual calculations from the partition table match the sizes reported by the operating system once units and file system overhead are taken into account.

10. Conclusion

In this lab I built a basic profile of a Kali Linux virtual machine without changing it. I identified the operating system, processor, memory and boot mode (BIOS/Legacy), looked at running kernel processes, and mapped how evidence is spread across the user, application, operating system, disk, memory and firmware layers.

I read the MBR partition table with fdisk and parted and calculated the sizes of the partitions from their sector counts. Partition 1 (sda1, ext4) is 28.42 GB and partition 2 (sda2, extended) is 1.58 GB, and these matched the sizes reported by the operating system after allowing for GB/GiB units and file system overhead.

This matters in digital forensics because an investigator must be able to prove, from the partition table and file system evidence, exactly what a disk contains and be able to repeat the same steps. Sector-level knowledge also shows where hidden or deleted data can remain,

such as unused gaps between partitions. Working with read-only commands protects the evidence so the results can be trusted.